

C# (.NET)

Colecciones e Interfaces

Colecciones

Hay dos tipos de colecciones: **genéricas y no genéricas**.

Las colecciones genéricas son aquellas en las que definimos en compilación el tipo de datos que contendrán, aceptan por tanto un parámetro de tipo cuando se construyen y no requieren conversiones con el tipo Object. Se agregaron en la versión 2.0 de .NET Framework

Las colecciones no genéricas almacenan los elementos como objeto (Admite todas las clases de la jerarquía de clases de .Net), requieren de conversión.

El espacio de nombres System.Collections.Generic expone muchas clases que pueden trabajar como contenedores genéricos de datos, elegiremos la más adecuada en función de para qué la necesitamos:

Lo que necesito	Genérica	No Genérica
Almacenar elementos como pares clave/valor para una consulta rápida por clave	Dictionary<TKey,TValue>	Hashtable
Almacenar elementos y acceder a ellos por su índice	List<T>	ArrayList
Utilizar elementos FIFO (el primero en entrar es el primero en salir)	Queue<T>	Queue
Utilizar datos LIFO (el último en entrar es el primero en salir)	Stack<T>	Stack
Acceder a los elementos de forma secuencial	LinkedList<T>	-----

Colección ordenada	SortedList<TKey,TValue>	SortedList
--------------------	-------------------------	------------

Todas las colecciones tienen métodos para agregar, buscar, eliminar elementos. Implementan *System.Collections.IEnumerable*, *System.Collections.IEnumerable<T>* que permiten procesar iteraciones sobre la colección.

Clase Stack (pila)

Para construir la estructura de una pila

```
Stack pila= New Stack(50);
```

Los tres métodos básicos son: Push , Pop y Peek

- Push: introduce un valor

```
pila.Push(5);
```

- pop : consulta la cima

```
pila.pop();
```

- Peek : extrae un elemento de la pila

```
pila.Peek ();
```

La propiedad Count proporciona el número de elementos

Clase Queue (Cola)

Para construir la estructura de una cola. Dispone de los siguientes métodos

- Enqueue : introduce un elemento
- Peek : consulta el primer elemento
- Dequeue: extrae el primer elemento
- Contains: comprueba si un elemento está en la cola
- Clear: elimina todos
- ElementAt (posicion) obtiene el elemento de esa posición

Clases List<T> y ArrayList

Los objetos de tipo colección creados con estas clases, implementan un array cuyo número de elementos puede modificarse dinámicamente. Se puede determinar o no de antemano su tamaño, si se crean sin tamaño por defecto se abren 16 elementos.

La clase ArrayList está definida en el namespace System.Collections mientras que la clase List lo está en el System.Collections.Generic

Los elementos de la colección ArrayList son de tipo Object por lo que podemos almacenar distintos tipos de objetos.

```
ArrayList lista= New ArrayList();  
ArrayList lista= New ArrayList(3);
```

Los elementos de la colección List están tipados

```
List<Alumno> lista=new List<Alumno>();  
List<Alumno> lista=new List<Alumno>(3);
```

Una vez creada una colección,

1. Para acceder a un elemento lista[posición]

2. Podemos utilizar algunos de los métodos indicados a continuación para **añadir** valores a la colección.

- **Add(Valor)**. Añade el valor representado por Valor.
- **AddRange(Colección)**. Añade un conjunto de valores mediante un objeto del interfaz ICollection, es decir, una colección dinámica creada en tiempo de ejecución.
- **Insert(Posición, Valor)**. Inserta el valor Valor en la posición Posición del array, desplazando el resto de valores una posición adelante.
- **InsertRange(Posición, Colección)**. Inserta un conjunto de valores mediante una colección dinámica, a partir de una posición determinada dentro del array.
- **SetRange(Posición, Colección)**. Sobrescribe elementos en un array con los valores de la colección Colección, comenzando en la posición Posición.

3. Count que devuelve el número de elementos que tiene el objeto

4. La colecciones nos proporcionan métodos para **obtener** un fragmento o rango (subarray)

- **GetRange(Posición, Elementos)**. Obtiene un subarray comenzando en el índice Posición, y tomando el número que indica Elementos.

5. Para **buscar** elementos: `IndexOf( )` y `LastIndexOf()`

6. Para realizar un **borrado** de valores proporcionan los métodos descritos a continuación.

- **Remove(Valor)**. Elimina el elemento de la colección que corresponde a Valor.
- **RemoveAt(Posición)**. Elimina el elemento situado en el índice Posición.
- **RemoveRange(Posición, Elementos)**. Elimina el conjunto de elementos indicados en el parámetro Elementos, comenzando por el índice Posición.
- **Clear()**. Elimina todos los elementos del objeto.

7. Al igual que en la clase base Array, estas colecciones disponen del método `Sort( )`, que **ordena** sus elementos implementando las interfaces *Comparable*, *Comparer* y del método `Reverse( )` que invierte las posiciones de un número determinado de elementos.

Comparable: La función de *Comparable* es proporcionar un método para comparar dos objetos de un tipo determinado. Esto es necesario si desea proporcionar alguna capacidad de ordenación para el objeto

Implement *Comparable* `CompareTo` method

Comparer: La función de *Comparer* es proporcionar mecanismos de comparación adicionales. Por ejemplo, puede desear proporcionar la ordenación de la clase en varios campos o propiedades

Tabla Hash

Una tabla Hash es una colección que almacena objetos asociando a cada uno de ellos una clave, por tanto guarda parejas (clave, valor) siendo valor un tipo básico o cualquier objeto.

A la hora de trabajar con los objetos almacenados no es necesario conocer su posición o sobrescribir métodos "equal" para poderlos buscar, es suficiente con especificar su clave

```
Hashtable ht = new Hashtable();
```

```
ht.Add( "clave", valor)  
ht["clave"] retorna el objeto correspondiente  
ht.ContainsValue(valor)  
ht.ContainsKey(clave)  
foreach (string k in ht.keys)
```

```
foreach(tipo variable in ht.Values)
```

Interfaces

Una interfaz no es más que una estructura de datos que muestra únicamente las firmas de los métodos de una clase. A partir de ahí, una clase que herede de la interfaz estará obligada a “rellenar” la implementación de dichos métodos

Las interfaces son la construcción que se encarga de disolver la ambigüedad producida por la herencia múltiple de clases. La siguiente figura muestra “el problema del diamante”. Este problema es una de las razones que justifica la omisión de la herencia múltiple en la mayoría de los lenguajes de programación

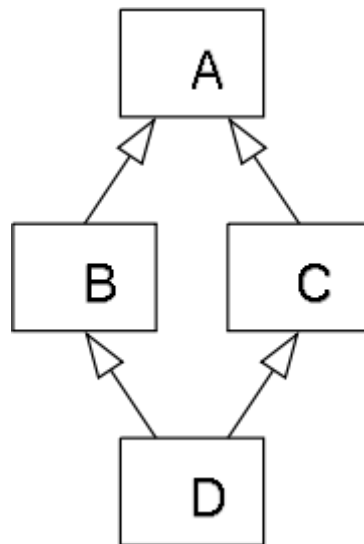


Figura 1. Problema del diamante

Evidentemente, para solucionar este problema sólo es posible practicar la herencia simple. Las interfaces pueden crear el efecto de herencia múltiple. Para ilustrar este concepto, veamos la Figura 2.

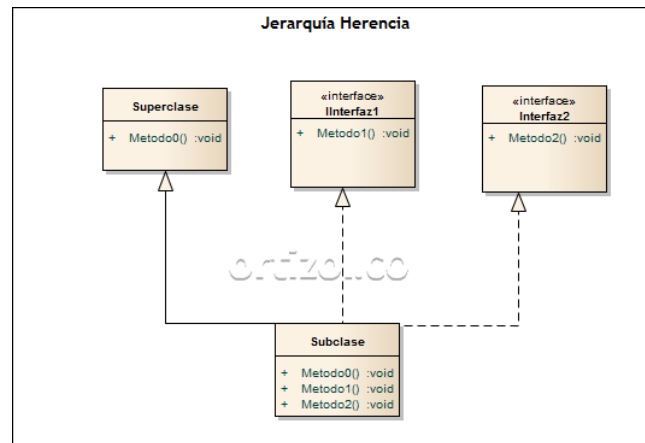


Figura 2. Herencia e implementación de interfaces.

Una interfaz es un protocolo de comunicación entre clases que no tienen por qué tener datos en común

Una interfaz es un tipo especial de clase que define la especificación de un conjunto de métodos. El conjunto de estos métodos garantizan que cualquier clase que implementa una interfaz tiene que proporcionar esos métodos.

Declaración de una interface:

```
interface nombre{
    declaración de métodos
}
```

Clase que implementa una interface:

```
class nombreclase : nombreI{
    cuerpos de los métodos del interface,
    datos y métodos propios
}
```

- Los métodos en la clase Interface no se implementan, solo se declaran.
- Si una clase implementa una interface debe codificar todos los métodos de la misma
- No puedo instanciar objetos de un interface con new
- Puedo asignar un objeto de una clase que implementa un interface a un objeto variable declarado de la clase interface.
- Un interface no puede tener campos, ni miembros estáticos, como mucho puede tener constantes.
- Una clase puede implementar múltiples interfaces, se separan con coma

Algunas Interfaces útiles

IComparable

La función de IComparable es proporcionar un método para comparar dos objetos de un tipo determinado. Esto es necesario si se quiere proporcionar alguna capacidad de ordenación para el objeto. Proporciona un criterio de ordenación predeterminado para los objetos. Por ejemplo, si tenemos una lista de objetos de un tipo, y se llama al método Sort de la clase List, IComparable proporciona la comparación de objetos durante la ordenación. Cuando se implementa la interfaz IComparable, se debe implementar el método CompareTo:

```
IComparable.CompareTo(object obj){  
    car c=(car)obj;  
    return String.Compare(this.make,c.make);  
}
```

La comparación en el método es diferente según el tipo de datos del valor que se va a comparar.

IComparer

La función de IComparer es proporcionar mecanismos de comparación adicionales. Podemos proporcionar la ordenación de la clase en varios campos o propiedades. El uso de IComparer es un proceso de algo más complejo